Day 68 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 6 reference

Watch: Bidirectional RNN | Deep Learning Tutorial 38 (Tensorflow, Keras & Python) (codebasics) | Deep RNNs | Stacked RNNs | Stacked LSTMs | Stacked GRUs (CampusX)

`day68_stacked_bidirectional.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula; the green **SYSTEM DIAGRAM** and the graph are the ones the script saved. A glossary is at the end.

MATH BOX LEGEND

Symbols: $\rightarrow h_t$ = left-to-right state at word t · $\leftarrow h_t$ = right-to-left state · [a ; b] = a and b joined side by side · R = reverse_padded, the flip of each sentence's real words · l = layer number (0 = bottom)

Segment 0 — Setup

Imports, paths and Day 67's settings

CODE

```
import copy
import os
import time
from collections import Counter, defaultdict
import numpy as np
from threadpoolctl import threadpool_limits
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
np.random.seed(42)
HERE = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(HERE, "..", "week-09-sequence-data-rnns-i", "ud_english_ewt_pos")
PAD, UNK = 0, 1
SEEDS = [0, 1, 2]
EPOCHS = 25
LR, FORGET_BIAS = 4.0, 1.0
```

- `LR, FORGET_BIAS = ...` — tuple unpacking: two names set on one line, to Day 67's validation choice for the LSTM.
- `LSTM and data functions` — copied verbatim from Days 62-67; not shown again here.

Segment 1 — Reversing padded sentences

reverse_padded

MATH

$$R(x)_t = x_{len-1-t} \quad \text{for } t < len, \quad x_t \text{ otherwise;} \quad R(R(x)) = x$$

CODE

```
def reverse_padded(A, lens):
    T = A.shape[1]
    t = np.arange(T)[None, :]
    idx = np.where(t < lens[:, None], lens[:, None] - 1 - t, t)
    if A.ndim == 3:
        idx = idx[:, :, None]
    return np.take_along_axis(A, idx, axis=1)
```

- `np.arange(T)[None, :]` — positions 0..T-1 as a (1, T) row, so it broadcasts against every sentence.
- `np.where(t < lens[:, None], lens[:, None] - 1 - t, t)` — real position t reads from position len-1-t; padding reads from itself.
- `idx[:, :, None]` — adds a last axis of size 1 so the same flip applies to every feature of a (m, T, D) array.
- `np.take_along_axis(A, idx, axis=1)` — gathers A[row, idx[row, t]] for every row and t in one call.
- `applied twice` — gives back the original, and it is its own gradient: the backward pass uses the same flip.

Segment 2 — The deep, bidirectional tagger

DeepTagger.__init__: one LSTM per layer per direction

CODE

```
class DeepTagger:
    def __init__(self, vocab_size, n_tags, layers=1, bidirectional=False, embed_dim=32,
        hidden_dim=32,
                 lr=LR, clip=1.0, seed=0, forget_bias=FORGET_BIAS):
        rng = np.random.RandomState(seed)
        self.layers, self.bi, self.n_tags, self.hidden_dim = layers, bidirectional, n_tags,
            hidden_dim
        self.lr, self.clip = lr, clip
        self.E = rng.randn(vocab_size, embed_dim) * 0.1
        self.directions = ["f", "b"] if bidirectional else ["f"]
        width = hidden_dim * len(self.directions)
        self.params = ["E"]
        for l in range(layers):
            in_dim = embed_dim if l == 0 else width
            for j, d in enumerate(self.directions):
                W, b = init_lstm(in_dim, hidden_dim, seed + 1000 * (2 * l + j), forget_bias)
                setattr(self, f"W{l}{d}", W); setattr(self, f"b{l}{d}", b)
                self.params += [f"W{l}{d}", f"b{l}{d}"]
        self.Why = rng.randn(width, n_tags) * np.sqrt(1.0 / width)
        self.by = np.zeros(n_tags)
        self.params += ["Why", "by"]
```

- `self.directions = ["f", "b"] if bidirectional else ["f"]` — a conditional expression picks one or two directions.
- `width = hidden_dim * len(self.directions)` — what a layer hands upward: 32 numbers per word, or 64 with two directions.
- `in_dim = embed_dim if l == 0 else width` — the bottom layer reads word vectors; higher layers read the layer below.

- `seed + 1000 * (2 * l + j)` — a different seed for every LSTM; layer 0's forward LSTM gets exactly Day 67's seed.
- `setattr(self, f"W{l}{d}", W)` — creates attributes named W0f, W0b, W1f, ... from strings.
- `self.params += [...]` — the list of names the training loop and the gradient check walk through.

forward: layers, and the second direction

MATH

$$\rightarrow H = LSTM_f(X); \quad \leftarrow H = R(\, LSTM_b(\, R(X)\,)\,); \quad H = [\, \rightarrow H\, ;\, \leftarrow H\,]$$

layer l + 1 reads H of layer l; tags come from the top layer's H

CODE

```
def forward(self, ids):
    m, T = ids.shape
    self.ids, self.lens = ids, (ids != PAD).sum(axis=1)
    h0 = np.zeros((m, self.hidden_dim))
    inp = self.E[ids]
    self.caches = []
    for l in range(self.layers):
        Hf, _, cf = lstm_forward(inp, h0, h0, getattr(self, f"W{l}f"), getattr(self, f"b{l}f"))
        caches = {"f": cf}
        if self.bi:
            Xr = reverse_padded(inp, self.lens)
            Hr, _, cb = lstm_forward(Xr, h0, h0, getattr(self, f"W{l}b"), getattr(self, f"b{l}b"))
            caches["b"] = cb
            Hf = np.concatenate([Hf, reverse_padded(Hr, self.lens)], axis=2)
        self.caches.append(caches)
        inp = Hf
    self.top = inp.reshape(m * T, -1)
    self.probs = softmax(self.top @ self.Why + self.by)
    return self.probs
```

- `(ids != PAD).sum(axis=1)` — each sentence's real length: only padding has id 0.
- `Xr = reverse_padded(inp, self.lens)` — the same input, every sentence written backwards.
- `reverse_padded(Hr, self.lens)` — the backward LSTM's states flipped back, so position t again means word t.
- `np.concatenate([Hf, ...], axis=2)` — join the two directions' states at every word: (m, T, 32) + (m, T, 32) -> (m, T, 64).
- `inp = Hf` — stacking in one line: this layer's output becomes the next layer's input.

gradients: splitting, flipping and adding

MATH

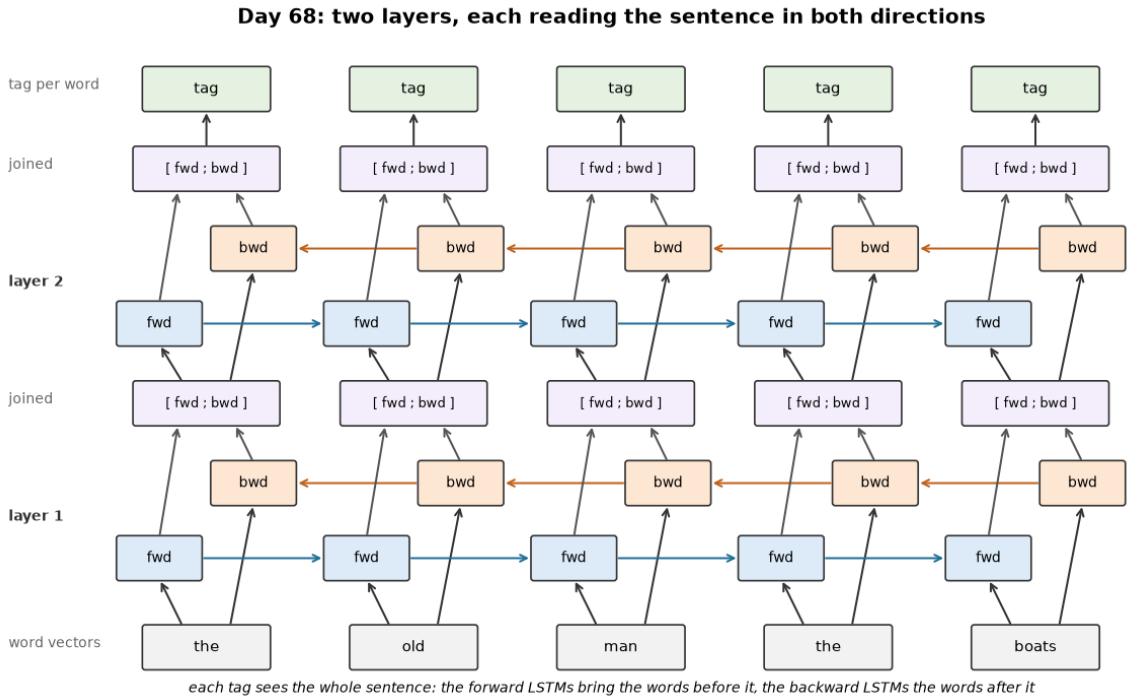
$$dX = dX_f + R(\, dX_b \,), \quad \text{where } dX_b \text{ comes from } LSTM_b\text{'s backward pass on } R(\, d{\leftarrow}H \,)$$

CODE

```
def gradients(self, tags, mask):
    m, T = self.ids.shape
    H = self.hidden_dim
    dlogits = (self.probs - one_hot(tags.ravel(), self.n_tags)) * mask.ravel()[:, None] /
        mask.sum()
    grads = dict(Why=self.top.T @ dlogits, by=dlogits.sum(axis=0))
    d_out = (dlogits @ self.Why.T).reshape(m, T, -1)
    zeros = np.zeros((m, H))
    for l in reversed(range(self.layers)):
        caches = self.caches[l]
        grads[f"W{l}f"], grads[f"b{l}f"], d_in, _, _ = lstm_backward(d_out[:, :, :H], zeros,
            zeros, caches["f"])
        if self.bi:
            dHr = reverse_padded(d_out[:, :, H:], self.lens)
            grads[f"W{l}b"], grads[f"b{l}b"], dXr, _, _ = lstm_backward(dHr, zeros, zeros,
                caches["b"])
            d_in = d_in + reverse_padded(dXr, self.lens)
        d_out = d_in
    grads["E"] = np.zeros_like(self.E)
    np.add.at(grads["E"], self.ids, d_out)
    return grads
```

- `d_out[:, :, :H], d_out[:, :, H:]` — the joined state's gradient splits back into its two halves.
- `dHr = reverse_padded(d_out[:, :, H:], self.lens)` — the backward LSTM ran on flipped sentences, so its gradient is flipped too.
- `d_in = d_in + reverse_padded(dXr, self.lens)` — the input fed BOTH directions, so its gradient is the sum of both.
- `d_out = d_in` — stacking, backwards: the gradient at a layer's input is the gradient at the output of the layer below.
- `for l in reversed(range(self.layers)):` — the top layer is processed first, the bottom layer last.

SYSTEM DIAGRAM



A 2-layer bidirectional tagger: forward states pass right, backward states pass left, and every tag sees both.

Segment 3 — Checks

The gradient check through two layers and two directions

CODE

```
net = DeepTagger(12, 3, layers=2, bidirectional=True, embed_dim=4, hidden_dim=5, clip=None,
    seed=1, forget_bias=0.5)
net.forward(ids)
analytic = net.gradients(tags, mask)
print(f"\n(b) {'parameter':>10}{'entries':>9}{'max relative error':>20}{'max
    |difference|':>18}{'pass':>7}")
ok_all = True
for name in net.params:
    numeric = numeric_gradient(net, name, ids, tags, mask)
    diff = np.abs(numeric - analytic[name])
    rel = diff / (np.abs(numeric) + np.abs(analytic[name]) + 1e-12)
    ok = bool(np.all(diff <= 1e-10 + 1e-6 * np.abs(numeric)))
    ok_all = ok_all and ok
    print(f"    {name:>10}{numeric.size:>9}{rel.max():>20.2e}{diff.max():>18.2e}{str(ok):>7}")
```

- `forget_bias=0.5` — a value other than 0 or 1, so the check does not rely on special numbers.
- `for name in net.params:` — all 11 arrays: E, four LSTMs' W and b, Why and by.
- `numeric_gradient(...)` — Day 62's central differences, unchanged.

Does the first word's tag depend on the last word?

CODE

```
changed = ids.copy(); changed[0, 4] = 3
p0 = net.forward(ids).reshape(4, 5, 3)[0, 0].copy()
p1 = net.forward(changed).reshape(4, 5, 3)[0, 0]
one = DeepTagger(12, 3, layers=2, bidirectional=False, embed_dim=4, hidden_dim=5, seed=1,
    forget_bias=0.5)
q0 = one.forward(ids).reshape(4, 5, 3)[0, 0].copy()
q1 = one.forward(changed).reshape(4, 5, 3)[0, 0]
```

- `changed[0, 4] = 3` — change only the last word of sentence 0.
- `reshape(4, 5, 3)[0, 0]` — back to (sentence, word, tag) and take sentence 0, word 0.
- `.copy()` — keep the first result: the next forward pass overwrites the network's arrays.

Segment 4 — Experiments

Training the four taggers and the size control

CODE

```
rows = CONFIGS[:1] + [("1 layer, left to right, h 50", dict(layers=1, bidirectional=False,
    hidden_dim=50))] + CONFIGS[1:]
for name, cfg in rows:
    accs, preds, epochs, trains = [], [], [], []
    for s in SEEDS:
        net, be, h = fit_cached(data, V, K, seed=s, **cfg)
        accs.append(h["test"][be]); epochs.append(be); trains.append(h["train"][-1])
        preds.append(net.predict(ids_te))
    results[name] = (accs, preds)
```

- `CONFIGS[:1] + [...] + CONFIGS[1:]` — list slicing and joining: the size control is inserted as the second row.
- `fit_cached(data, V, K, seed=s, **cfg)` — the settings dictionary is unpacked into keyword arguments.
- `'/'.join(str(e) for e in epochs)` — three best epochs printed as one string like 12/9/14.

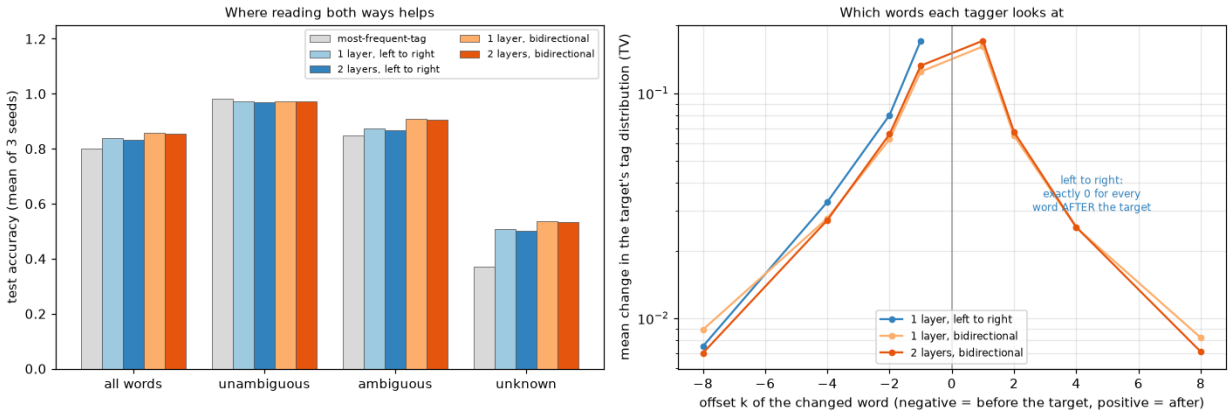
The probe in both directions

CODE

```
rows = np.where(lens > abs(k))[0]
draws = []
for _ in range(5):
    for r in rows:
        t = rng.randint(max(0, -k), lens[r] - max(0, k))
        draws.append((r, t, rng.randint(2, V)))
draws = np.array(draws)
r_idx, t_idx, new_w = draws[:, 0], draws[:, 1], draws[:, 2]
changed = ids_te[r_idx].copy()
changed[np.arange(len(draws)), t_idx + k] = new_w
keep = changed[np.arange(len(draws)), t_idx + k] != ids_te[r_idx, t_idx + k]
```

- `rng.randint(max(0, -k), lens[r] - max(0, k))` — a target position that leaves room for a word k before it (k < 0) or after it (k > 0).
- `changed[..., t_idx + k] = new_w` — the edited word is now on EITHER side of the target.
- `!=` — keeps only draws where the replacement really differs from the original word.

REAL GRAPH



Real graph from the script: accuracy by word group, and which words each tagger looks at.

Quick Syntax Glossary — new or reinforced constructs

- `np.take_along_axis(A, idx, axis=1)` — gather along one axis with a different index for every row.
- `np.where(cond, a, b)` — a where cond is true, b elsewhere, element by element.
- `setattr(obj, name, value)` — create or set an attribute whose name is a string.
- `f"W{l}{d}"` — an f-string building a name from variables.
- `np.concatenate([A, B], axis=2)` — join two arrays along their last axis.
- `reversed(range(n))` — n-1, n-2, ..., 0.
- `a = b if cond else c` — a conditional expression.
- `dict(CONFIGS)` — a list of (key, value) pairs turned into a dictionary.

End of Day 68 syntax reference. For the experiments and what they showed, see the Day 68 complete guide.